# COGNIFORGE: A Verified Multi-Agent Architecture for Contingency-Aware, Audit-Traceable Robot Teleoperation in WebXR

Department of Automation and Robotics Engineering
*Bachelor of Technology Final Year Project*

## Abstract

Robot programming by demonstration promises to remove the programming-language barrier between domain experts and industrial manipulators, but existing systems either require specialized physical instrumentation or reduce the human demonstrator to a single, unbranching hypothesis about intent. This paper presents COGNIFORGE, a browser-based WebXR platform in which an operator's hand motion is interpreted by a nine-node cooperative agent pipeline combining reactive safety reflexes, Belief-Desire-Intention (BDI) planning, contingency-game-style multi-hypothesis motion planning resolved through a real Damped Least Squares inverse-kinematics solver, and a dual-path (analytic and PyBullet) collision validator, with every agent decision recorded in a hash-chained, tamper-evident audit ledger. Distinct from the project's original proposal, which specified a five-stage offline learning-from-demonstration pipeline, the delivered system additionally supports live, frame-by-frame shared control with branching trajectory previews the operator can approve or override in real time. We report a systematic verification process that surfaced and fixed ten concrete defects -- from a silent schema mismatch that disabled the entire agent pipeline to platform-specific crashes uncovered only by packaging the application as a native desktop and mobile shell -- and an automated benchmark harness that replaces the proposal's pre-registered but never-collected human-subject numbers with real, reproducible measurements against the running system. The same WebXR codebase is packaged three ways: as a browser application, as an Electron desktop application (verified end-to-end via the Chrome DevTools Protocol), and as a Capacitor-based Android application compiled to a verified, installable APK, sideloadable directly onto a standalone Meta Quest headset for immersive VR. A second verification pass on physical Android hardware -- absent from the primary development environment and conducted through a remote testing protocol detailed in Section 5.1 -- found that Capacitor's embedded WebView cannot execute WebXR's immersive-ar session at all, a platform limitation not evident from any documentation consulted during initial development; the mobile AR experience was consequently re-implemented as a native ARCore Android application with a hand-written, minimal OpenGL ES renderer, trading the original CAD-mesh visual fidelity for arm geometry whose correctness is verifiable by construction.

## Keywords

Multi-Agent Systems, WebXR, Learning from Demonstration, Contingency Games, Belief-Desire-Intention, Inverse Kinematics, Human-Robot Interaction, Software Verification, Audit Logging, Cross-Platform Deployment.

## 1. Introduction

### 1.1 Motivation

Industrial and collaborative robots remain difficult for non-specialists to program. Code-based programming demands fluency in vendor-specific languages; teach pendants scale poorly to multi-step tasks; kinesthetic teaching requires unsupervised physical access to the robot. A parallel and increasingly urgent concern, drawn from the multi-agent security literature this project surveyed prior to implementation, is that individually reasonable agent decisions can still compose into an unsafe or unauditable system -- a

non-compositionality problem that argues for verifiable, logged decision trails wherever multiple autonomous components jointly control a physical actuator.

This project began from a proposal describing a five-agent (Perception, Motion Planning, Error Correction, Memory, Meta-supervisor) offline learning-from-demonstration pipeline. During implementation, the system was extended toward a richer, live shared-control architecture informed by three additional bodies of research reviewed alongside the original proposal: contingency-game motion planning for multi-hypothesis human intent (Fridovich-Keil et al.-style branching trajectories), BDI cognitive architectures for long-horizon rational planning, and the emerging Multi-Agent Security (MASEC) literature's emphasis on tamper-evident decision auditing. The result is a system that both processes recorded demonstrations in the proposal's original batch sense and reacts to a live operator frame-by-frame, sharing one robotics core (kinematics, trajectory smoothing, collision validation) between the two modes.

## 1.2 Contributions

- A nine-node LangGraph agent pipeline unifying reactive safety, BDI deliberation, contingency-branch motion planning, and rule-plus-simulation error correction, replacing an earlier version of the codebase in which the motion-planning stage did not call the system's own inverse-kinematics solver at all.
- A hash-chained audit ledger (SQLite-backed, SHA-256, each entry embedding the previous entry's hash) that makes every agent decision independently verifiable, consolidating two previously divergent and mutually inconsistent implementations found in the codebase at the start of this work.
- A documented, reproducible verification process that found and fixed ten concrete defects spanning backend logic, live browser interaction, and cross-platform packaging -- reported in full in Section 5 as a methodological contribution in its own right.
- An automated evaluation harness that exercises the real backend pipeline (real IK, real smoothing, real collision validation) against synthetic demonstrations, replacing placeholder figures with measured ones and explicitly marking the one proposal metric (User Effort Score) that genuinely requires a human-subject study never conducted here; the same five task definitions it benchmarks are also exposed live (GET /tasks) so every deployed app can play any one of them on demand, rather than the harness and the interactive system risking two different definitions of the same task.
- A single WebXR codebase packaged three ways -- browser, Electron desktop, and a compiled, verified Android application -- confirmed to actually render and connect to the backend in each shell, not merely scaffolded.
- A native ARCore Android AR implementation, built after a second verification pass on physical hardware established that WebXR's immersive-ar session cannot execute inside Capacitor's embedded WebView at all -- a platform-level finding reported in Section 5.1 together with the remote-testing methodology used to diagnose it without direct access to the failing device.

## 1.3 Development Timeline

This work is organized below by development phase rather than calendar date. Version control for this project preserves only three widely-spaced commit dates rather than a granular week-by-week record, so a phase-based account is reported here instead of a reconstructed schedule; each phase is detailed in full, including the specific defects found and fixed within it, in the section noted.

| Milestone | Description | Reported in |
|---|---|---|
| M1 -- Requirements & Proposal | Original scope: a five-agent offline learning-from-demonstration pipeline, to be evaluated via a ten-participant Meta Quest 3 human-subject study. | Section 1.1 |

| | | |
|---|---|---|
| M2 -- Literature Grounding | Reviewed contingency-game motion planning, BDI cognitive architectures, and multi-agent security (MASEC) research, reshaping the architecture toward live shared control. | Section 2 |
| M3 -- Core Architecture & Kinematics | Nine-node LangGraph pipeline, real Damped Least Squares inverse kinematics, dual-path collision validation, hash-chained audit ledger. | Section 3 |
| M4 -- Multi-Platform Packaging | One WebXR codebase split into a browser/Electron desktop build, a Capacitor VR build, and an initial Capacitor/WebXR AR build. | Section 4 |
| M5 -- Verification Pass 1 (Web/Desktop) | Static checks, automated tests, and live interactive testing found and fixed ten concrete defects. | Section 5 |
| M6 -- AR Discovery & Native Rewrite | A second verification pass on physical hardware found the WebXR/Capacitor AR approach could not run at all; rebuilt as a native ARCore application, fixing six further defects. | Section 5.1 |
| M7 -- Evaluation | Automated benchmark harness measuring the real pipeline against all five task categories, replacing the proposal's never-collected placeholder figures. | Section 6 |
| M8 -- Live Demonstrability & Documentation | Added live, on-demand task-scenario playback and an operator-facing audit-ledger view; finalized this paper, the presentation, and a cross-platform build guide. | Section 4 |

Phases are listed in the order they were completed, and each depended on the one before it: the verification methodology in M5, for instance, could not have surfaced live-interaction defects before M4's packaging work gave it a running application to interact with.

## 2. Related Work

Argall et al. survey the foundational challenges of learning from demonstration: policy derivation, state-space correspondence between teacher and learner, and performance-metric design [1]. Niekum et al. demonstrate unstructured-demonstration segmentation via a beta-process autoregressive hidden Markov model, but depend on physical, marker-based instrumentation [2]. Calinon et al. encode demonstrated trajectories with Gaussian Mixture Models and reproduce them via Gaussian Mixture Regression, informing this project's use of smoothing and warm-starting in the Motion Planning stage [3].

On the interaction side, Lipton et al. show that virtual reality improves teleoperation ergonomics and situational awareness relative to 2D interfaces [4], and Rosen et al. demonstrate augmented-reality robot programming, though at the task level rather than full kinematic demonstration [5]. None of these systems combine WebXR-based capture with a multi-agent interpretation pipeline, a gap this project's original proposal identified and which the delivered system still addresses, now via a richer agent topology than originally scoped.

The delivered architecture's contingency-branch motion planning is conceptually aligned with game-theoretic motion planning under intent uncertainty, in which a planner commits to a common trajectory trunk while preparing separate branches per hypothesis until uncertainty resolves. The BDI (Belief-Desire-Intention) agent module follows the classical practical-reasoning cycle -- perceive, update beliefs, generate applicable plans, select and commit to an intention -- as formalized in AgentSpeak(L)-style cognitive architectures [14]. Multi-agent systems research more broadly (Stone and Veloso; Parker's ALLIANCE architecture) motivates this project's Meta-Agent supervisor, which performs the same fault-detection and escalation role ALLIANCE assigns to motivational behaviors, adapted here to a software-agent context with a bounded retry counter rather than unbounded re-negotiation.

Finally, recent multi-agent security scholarship establishes that securing each agent in isolation is necessary but not sufficient: composed systems can exhibit emergent unsafe behavior even when no single component is individually flawed. This motivated the hash-chained audit ledger described in Section 3.4 -- a lightweight, blockchain-inspired mechanism for making the full decision trail independently verifiable after the fact, rather than trusting any single agent's self-report.

## 3. System Architecture

COGNIFORGE retains the proposal's three-tier separation -- a WebXR Presentation Layer, a Multi-Agent Intelligence Layer, and a Robot Execution Layer -- communicating over WebSocket (live streaming) and REST (session management). Within the Intelligence Layer, the delivered pipeline diverges substantially from the original five-agent specification.

### 3.1 The Nine-Node Agent Pipeline

Incoming WebXR telemetry first reaches a Gateway agent, which validates and sanitizes each frame -- rejecting, for example, joint positions outside a bounded coordinate range as a defense against malformed or adversarial input. A Perception agent then classifies grasp gestures (pinch detection via fingertip distance) and segments motion into IDLE/MOVING primitives via a velocity threshold over a short position history. A Visual Reasoning agent maintains the scene graph passed alongside telemetry (workspace boundary, world-in-miniature widget) for downstream context.

An Intent Prediction agent generates multiple weighted hypotheses about where the operator's hand is heading -- the contingency-game element of the design -- which a Reactive agent checks first, unconditionally, against a fixed safety radius around the robot's physical base, issuing an immediate emergency stop before any deliberative reasoning executes if the operator's hand enters that radius. If no emergency applies, a BDI Planning agent updates a small belief store (most-likely intent) and commits to an intention (for example, 'Assist Handover' when the dominant hypothesis is a forward reach).

A Motion Planning agent then resolves the primary intent hypothesis, and each remaining hypothesis as a contingency branch, to real joint angles via Damped Least Squares inverse kinematics (Section 3.2) -- optionally warm-started from a similar prior demonstration retrieved by the Memory agent. An Error Correction agent validates the resulting waypoint against workspace and collision constraints (Section 3.3), classifying any violation as auto-correctable or requiring operator review, and re-invoking Motion Planning with a corrective nudge up to a bounded number of attempts before escalating to the operator rather than looping indefinitely. Finally, a Meta-Agent finalizes the pipeline's decision, and on a successful execution persists the resulting trajectory into the skill library for future retrieval.

### 3.2 Inverse Kinematics

The Motion Planning agent uses a Damped Least Squares (DLS) solver over a six-degree-of-freedom Denavit-Hartenberg model approximating a UR5-class arm: [16]

$$\Delta q = J^T (J J^T + \lambda^2 I)^{-1} e$$

where J is the numerically differentiated Jacobian, e is the Cartesian position error, and λ is a fixed damping factor that keeps the update well-conditioned near kinematic singularities -- a property a naive Jacobian-transpose or pseudo-inverse update lacks [15]. This solver replaced an earlier implementation in which the motion-planning stage never called any inverse-kinematics routine at all, instead writing the operator's raw hand coordinates directly into the robot's joint-angle array (Section 5, defect 2).

### 3.3 Collision Validation

The Error Correction agent's geometric safety check runs against a CollisionValidator that prefers a headless PyBullet physics client when available -- approximating the arm as a capsule chain and querying closest-point contacts against a ground plane and against non-adjacent links for self-collision -- and falls back to analytic floor-clearance and workspace-radius checks when PyBullet is unavailable. PyBullet could not be installed in this project's development environment: its bundled third-party zlib shim conflicts with the macOS 26 SDK's declaration of fdopen() under the Xcode Command Line Tools available here, an upstream compatibility issue rather than a defect in this codebase. Both code paths share the same interface, so the pipeline's behavior degrades gracefully rather than failing outright; enabling the PyBullet path on a compatible platform is listed as future work.

### 3.4 Memory and Audit

A three-tier hierarchical graph memory (interaction graph, query graph, insight graph) records cross-agent collaboration traces and distills them into persisted insights, following the structure of recent hierarchical multi-agent memory research. A ChromaDB-backed skill library stores a text description and trajectory signature for every completed session, enabling semantic retrieval to warm-start inverse kinematics for related future tasks. [18]

Independently, a SecureAuditor logs every agent decision to a SQLite-backed ledger in which each row's hash is computed over its own content concatenated with the previous row's hash -- a minimal blockchain-style construction [17] that makes tampering with any historical entry detectable by replaying the chain (verify_integrity()). At the start of this project, the codebase contained two divergent implementations of this idea under the same class name: one (in active use) that hashed each entry independently without chaining to its predecessor, and a second, unused implementation that chained correctly. These were consolidated into the single implementation described here, and a unit test now asserts that mutating any historical row is detected.

## 4. Multi-Platform Implementation

The WebXR frontend (React, Three.js, the WebXR Device API and Hand Input Module, with a MediaPipe-based head-tracking fallback for non-headset use) was refactored during this work from a single 553-line Engine class into dedicated modules -- a WebSocketClient handling connection lifecycle and reconnection, and a HandTracking module handling joint serialization and the desktop mouse fallback -- matching the modular architecture the project's own documentation described but had not, at the start of this work, actually implemented (the corresponding source directories existed but were empty).

Beyond the browser, the same production build is packaged as an Electron desktop application and as a Capacitor Android/iOS application. Both required a Vite configuration change (relative rather than absolute asset paths) to load correctly outside an http(s) origin, and both surfaced a further defect -- a WebSocket client that read window.location.hostname (an empty string under file://) with no fallback, throwing a synchronous, uncaught exception that prevented the entire React application from mounting -- found only by connecting to the packaged Electron build via the Chrome DevTools Protocol and observing an empty root element with zero rendered DOM nodes. The mobile build additionally gains a WebXR immersive-ar session path (hit-test-based surface placement for positioning the virtual workspace in a real room) alongside the existing immersive-vr path, so the identical build can run as VR on a sideloaded standalone Quest install or as handheld AR on an ARCore-capable phone, selected automatically by runtime feature detection rather than by separate builds.

The Android APK was subsequently compiled and verified: a command-line-only toolchain (Android SDK Command-Line Tools, OpenJDK 21, Gradle 8.14.3, Android Gradle Plugin 8.13.0) was installed to avoid the disk footprint of a full Android Studio installation, and the resulting debug APK was verified by

decoding its manifest (aapt dump badging) and inspecting its packaged assets directly rather than trusting the build's exit code alone. Two Android applications were built from this pipeline: a VR-only Capacitor application unchanged from the WebXR codebase described above, and a second Android application -- initially also a Capacitor/WebXR build -- for handheld AR, whose divergent final architecture is the subject of Section 5.1. An iOS build was not attempted, consistent with the iOS WebXR limitation noted in Section 8.

A further capability was added after initial packaging: the same five task scenarios Section 6's benchmark harness exercises are now playable on demand inside all three deployed applications, not only offline. A GET /tasks endpoint serves app.robotics.tasks.TASKS -- the identical list the harness already imported, refactored into a single canonical definition so the two paths cannot silently diverge -- and each app resolves it independently: the WebXR-based browser/desktop/VR build sequentially requests /solve_ik for each waypoint and interpolates the rendered arm between the returned joint poses, while the native ARCore application reimplements the identical waypoint-chaining animation in Kotlin, since it shares no code with the WebXR build (Section 5.1). Selecting a task no longer requires reproducing its trajectory by hand with a mouse or headset controller; the operator picks one, and the arm executes the real IK-solved motion autonomously while the audience watches.

## 5. Verification: A Case Study in Ten Defects

Because this system composes multiple autonomous agents controlling a (simulated) physical actuator, correctness could not be established by code review alone. The verification pass documented here combined automated testing (23 backend tests covering kinematics, trajectory smoothing, collision validation, the hash-chained ledger, and full pipeline smoke tests including a regression test for each defect below), TypeScript static checking and a production build for both the web and native shells, and live, interactive browser verification of the actual running system -- which is what surfaced the majority of the defects in Table 1, none of which static analysis or a passing unit-test suite alone would have caught.

| # | Defect | Detection method | Impact if unfixed |
|---|---|---|---|
| 1 | Frame schema field-name mismatch (camelCase JS vs snake_case Python, no alias) | Automated pipeline smoke test | Every WebXR frame rejected at the Gateway node; the entire agent pipeline never executed |
| 2 | Motion planner fed raw hand XYZ directly into the joint-angle array instead of solving IK | Code audit + kinematics unit tests | Robot visualization moved to physically meaningless poses; no real inverse kinematics occurred |
| 3 | Safety-reflex used a raw z-axis threshold uncalibrated to the scene's actual robot-base position | Regression test after the schema fix exposed the pipeline for the first time | Emergency-stop would misfire on safe poses (e.g. the desktop mouse-fallback path) and fail to protect true near-base positions |
| 4 | Duplicate, conflicting memory and audit-ledger implementations under the same class name | Manual code audit | Non-deterministic behavior depending on import order; audit ledger not actually hash-chained |
| 5 | React effect re-mounted the entire WebGL engine and WebSocket connection on every HITL flag change | Live browser verification via automated interaction | Operator approval appeared to have no effect; scene and connection state reset on every contingency |
| 6 | Natively-injected VRButton DOM element intercepted clicks on an overlapping React-rendered panel | Live browser verification (elementFromPoint diagnostic) | New session-recording UI was unusable despite rendering correctly |
| 7 | Unhandled fetch/WebSocket errors left the UI stuck in a non-terminal “finalizing” state | Live browser verification of the error path | A short demonstration (too few waypoints) would hang the interface indefinitely |
| 8 | list-index-out-of-range crash on a | Live browser verification | Any Gateway-rejected frame silently |

| | | | |
|---|---|---|---|
| | present-but-empty trajectory list | during HITL testing | dropped the WebSocket connection |
| 9 | Absolute Vite asset paths and an empty location.hostname under file:// | Chrome DevTools Protocol inspection of the packaged Electron build | The desktop and Capacitor mobile builds rendered a blank white/black screen with no visible error |
| 10 | Audit-ledger hash chain forked by an unsynchronized read-then-write race when two processes (the live dev server and the offline benchmark harness) wrote to the same SQLite-backed ledger concurrently | Live integrity check exposed through an operator-facing ledger panel built to surface agent activity during demonstrations, while two independent processes held the same ledger file open at once | Ordinary concurrent development or testing activity -- not malicious tampering -- could fork the chain and trip the tamper-evidence check, falsely branding a legitimate ledger as compromised and undermining the audit trail's central guarantee |

*Table 1: Defects found and fixed during verification, in the order encountered.*

Defects 1-4 were found through backend code audit and automated testing; defects 5-8 were found only by actually driving the running application in a browser and observing behavior that contradicted the intended design (for example, clicking an 'Approve' button that appeared to have no effect); defect 9 was found only by packaging and inspecting the native desktop build, and would have shipped silently in the mobile build as well had it not been caught at that stage; defect 10 surfaced only once two independent processes were operating against the system's SQLite-backed audit ledger at the same time, a failure mode no single-process test, however thorough, could have exercised. This progression -- static checks, automated tests, live interactive use, native packaging, then sustained concurrent operation -- is offered as a template for verifying agentic systems with a real-world actuation surface, where each layer catches a class of defect invisible to the layers before it. Defect 10 was fixed by serializing the ledger's read-last-hash-then-insert sequence behind a process-local lock and a SQLite BEGIN IMMEDIATE transaction, so two writers can no longer observe the same prior hash.

## 5.1 A Second Case Study: Native Mobile AR Verification

The verification methodology of Section 5 assumes the verifier can directly operate the running system. Confirming the Android AR build required relaxing that assumption: the primary development environment has no ARCore-capable device, so the mobile build was instrumented with on-screen diagnostic logging in place of an attached debugger or device log stream, and verified through an iterative remote-testing protocol in which a tester on a personal Android device exchanged screenshots and log excerpts. Single-variable isolation tests -- for example, temporarily replacing the render clear color with a value no correct frame would ever produce, to determine whether GPU output was reaching the display at all, independent of whether the camera texture or 3D content were correct -- substituted for the interactive breakpoint debugging Section 5's case study relied on.

This process surfaced a finding more fundamental than any single defect: WebXR's immersive-ar session mode, which the original mobile build (Section 4) assumed would behave like immersive-vr inside a Capacitor WebView, could not be started at all on the test device, reporting unsupported regardless of manifest permissions, ARCore installation state, or renderer configuration. The camera-permission and manifest fixes applied first (Table 2, rows 1-2) were necessary but insufficient corrections to a symptom rather than the underlying cause: Capacitor's Android shell embeds a plain android.webkit.WebView, and WebXR camera-passthrough sessions are understood to require the full Chrome application process, or a Trusted Web Activity wrapping it, rather than a generic embedded WebView instance. No configuration change inside the existing architecture could resolve this, so the AR build was re-implemented directly against the ARCore SDK [19] (v1.54.0) with a hand-written, minimal OpenGL ES 2.0 renderer, deliberately avoiding third-party scene-graph wrapper libraries whose API surface at any specific version could not be validated without the same interactive build-test loop this section's methodology was built to work around.

Table 2 reports the defects found while bringing the native rewrite to a working state, using the same detection-method/impact format as Table 1. As in Section 5, several of these (rows 3-6) were invisible to any pre-deployment check: they manifested only as an unexplained absence of expected output on the physical device, with the underlying cause established by systematic elimination rather than by an error message.

| # | Defect | Detection method | Impact if unfixed |
|---|---|---|---|
| 1 | Missing CAMERA runtime permission and ARCore manifest declarations | Code audit against ARCore/Android documentation after an initial "AR not supported" report from the tester | Camera access and hit-testing could never initialize; every AR session request failed before reaching any application code |
| 2 | Android's default HTTP cleartext-traffic block (no manifest opt-in declared) | Reading the specific exception surfaced by an improved error handler, added after a generic failure message gave no actionable detail | Every backend request from the device failed regardless of a correct IP address, indistinguishable from a genuine network misconfiguration without the improved logging |
| 3 | Camera-texture output not reaching the display despite the render pipeline reporting no GL error | Single-variable isolation test: temporarily replacing the frame clear color with an implausible value to separate "GPU output not reaching the display" from "wrong output being produced" | Camera passthrough appeared entirely non-functional with no signal distinguishing a rendering-pipeline fault from a display-compositing fault |
| 4 | Opaque window-level background painted over the GLSurfaceView's compositing surface | Root-caused once isolated by defect 3, matching a documented interaction between SurfaceView-backed views and an opaque android:windowBackground | Camera passthrough would have remained invisible regardless of any further shader or texture correction, since the fault sat one layer above the rendering code |
| 5 | World anchor implemented as a one-time pose snapshot rather than a live ARCore Anchor | Reported by the tester as the placed object visibly drifting from its real-world position during ordinary use | The core AR interaction -- an arm that appears fixed in the room -- would not hold up under the device motion AR is meant to support |
| 6 | CAD-mesh joint-chain rendering as visually disconnected segments across every tested viewing angle, despite a numerically correct transform chain | The joint-transform chain was independently re-verified in a standalone script outside the Android build, confirming the mathematics and isolating the fault to unverifiable mesh-authoring assumptions | An arm that is kinematically correct but does not read as one connected robot undermines the core demonstration independent of any other correctness property |

*Table 2: Defects found while verifying the native ARCore Android implementation on physical hardware not available in the primary development environment.*

Row 6 was resolved not by finding the specific authoring assumption the loaded CAD meshes violated, but by removing the assumption altogether: the arm's geometry was replaced with cylinders and spheres generated directly by the rendering code, with each link's connecting cylinder constructed to terminate at exactly the coordinate the next joint's housing is centered on, by shared construction rather than by two independently authored values that happen to agree. This trades the photorealistic detail of the original CAD import for a correctness property provable from the transform chain alone -- a deliberate choice given that the fault it replaced could not be visually diagnosed without the interactive tooling available in Section 5's browser-based case study, and is reported here as a second methodological point: when direct inspection

of a rendering fault is unavailable, restructuring an implementation so correctness follows from its construction can be preferable to continuing to search for a defect in inspectable but unverifiable third-party input data.

A related cleanup followed once the procedural rewrite made the original Capacitor WebView shell (MainActivity) entirely unreachable: it had no intent-filter and was not exported, so nothing could launch it, yet the roughly 11MB WebView asset bundle -- the pre-rewrite JS bundle and the same CAD meshes Row 6 replaced -- that Capacitor's sync step copies into the Android build kept being repackaged into every .apk regardless. Excluding that bundle at the Android build layer, while leaving it untouched for the iOS build (which still depends on it for a WebView-based head-tracking fallback in the absence of any ARCore equivalent), cut the shipped APK from 10.6MB to 7.9MB with no behavior change, alongside removing the now-dead MainActivity class and its OBJ/MTL mesh parser.

## 6. Evaluation

The project proposal specified a ten-participant human-subject study using a Meta Quest 3 headset. That study was not conducted -- no headset or human participants were available in this project's development environment -- and this paper does not reuse the proposal's pre-registered placeholder figures (87.8% TCR, 3.7s DEL, 0.91 TFS) as if they were results. Instead, an automated harness (backend/evaluation/) exercises the real, running backend pipeline against synthetic demonstrations: for each of the proposal's five task categories, ten trials add Gaussian hand-tremor noise ($\sigma$ = 1 cm) to an ideal Cartesian trajectory, resolve it through the actual DLS inverse-kinematics solver, and push it through the real session-based REST pipeline, including real cubic-spline smoothing and real collision validation. These same five task definitions are also playable live in every deployed application (Section 4), so this table's numbers and what a viewer can watch happen on screen are drawn from one definition, not two.

| Task | TCR (%) | DEL (s) | TFS | n |
|---|---|---|---|---|
| Pick-and-Place | 100.0 | 0.0253 | 0.983 | 10 |
| Stacking | 100.0 | 0.0585 | 0.982 | 10 |
| Path Tracing | 100.0 | 0.0623 | 0.983 | 10 |
| Assembly (2-Part) | 100.0 | 0.0416 | 0.981 | 10 |
| Assembly (Complex) | 100.0 | 0.0115 | 0.976 | 10 |

*Table 3: Automated benchmark results (n = 10 trials per task; see Section 6.1 for metric definitions and scope).*

### 6.1 Metric Definitions and Honest Scope

Task Completion Rate (TCR) here measures the fraction of trials the collision validator accepts, not physical-world task success. Demonstration-to-Execution Latency (DEL) is measured as in-process finalize-endpoint wall-clock time and excludes the network, WebSocket, and device-side latency the original proposal's figure was meant to include. Trajectory Fidelity Score (TFS) is computed as one minus the forward-kinematics error between the noiseless ideal target and the achieved final smoothed pose, normalized by the workspace radius -- it measures whether noisy input still resolves to the intended target, not similarity between a demonstrated and executed path. Error Correction Accuracy was measured separately against three deliberately infeasible (below-floor) probe trajectories, all three of which the collision validator correctly rejected (100%); three probes is a small sample and this is reported as a sanity check rather than a robust estimate. User Effort Score is not reported: it is a human-factors metric that requires participants, and none were available.

Table 3 reports results from re-running this harness immediately before submission, rather than an earlier pass kept for the record. Repeated runs produce slightly different Demonstration-to-Execution Latency figures from run to run -- for example, an earlier pass measured 12.1ms for Pick-and-Place against the 25.3ms reported here -- because each trial's synthetic hand-tremor noise is drawn fresh from an unseeded

Gaussian distribution (Section 6's sigma = 1cm), so the exact starting point the DLS solver iterates from, and therefore its exact iteration count, varies between runs even though the underlying pipeline code is unchanged. Trajectory Fidelity Score is markedly more stable across runs than latency is (differing by at most 0.001 across every task observed), consistent with TFS measuring whether the solver converges to the correct target at all rather than how many iterations that convergence took. This run-to-run variance is disclosed here rather than masked by fixing a random seed, since a fixed seed would make the harness deterministic in a way the system it measures is not, and the harness source (backend/evaluation/) is checked into the repository for independent re-verification.

The uniform 100% TCR reflects that the five task trajectories were deliberately specified inside the arm's safe, reachable workspace -- this demonstrates pipeline correctness on well-posed inputs, not robustness under harder conditions such as near-boundary targets or tighter collision tolerances. A stress-test extension of this harness (adversarial targets closer to the workspace boundary, tighter tolerances, and a larger adversarial sample) is listed as future work.

## 7. Discussion

The gap between this project's original proposal and its delivered implementation is itself informative. The proposal specified a offline, batch-oriented five-agent LfD pipeline; the delivered system instead centers on a live, frame-by-frame shared-control loop with contingency branching, wrapping the original LfD flow as a session-based recording mode built on the same robotics core. This shift emerged from engaging with a broader literature (contingency games, BDI architectures, multi-agent security) during implementation rather than being planned upfront, and produced a system that is arguably more novel -- unifying reactive safety, deliberative planning, and multi-hypothesis motion planning behind a single tamper-evident audit trail -- than the originally scoped architecture, at the cost of the original proposal's five-agent naming no longer describing the running system precisely. We consider this an honest account of how the project actually evolved rather than retrofitting the paper's narrative to match superseded planning documents.

The verification case study in Section 5 also argues for a broader methodological point relevant to multi-agent robotics research generally: several of the ten defects found here (in particular defects 5-10) are structurally invisible to unit testing and static analysis, because they arise from the interaction between a framework's lifecycle (React's effect re-execution), a library's DOM injection behavior (Three.js's VRButton), a runtime environment's URL semantics (file:// versus http://), and -- for defect 10 -- the relative timing of two independent operating-system processes, none of which any single component's own tests would exercise. Live, interactive verification of the actually running system -- and, for cross-platform claims, actually launching the packaged native shell -- surfaced defects that would otherwise have shipped.

Section 5.1's mobile AR case study extends this argument one step further: some properties of a system -- specifically, whether a given host environment can execute a given API at all -- are invisible not just to static analysis, but to the developer's own interactive testing if that testing never leaves the development machine. WebXR's immersive-ar session behaved identically to immersive-vr in every environment available during initial development and diverged only on a physical device neither available in, nor represented by, that environment; no amount of additional live testing within the original development setup could have surfaced this. This suggests a corollary to Section 5's methodological claim: for cross-platform claims specifically, "live verification" must include the actual target hardware, not merely the actual target software stack running on whatever hardware happens to be convenient.

A note on what “simulation” means for this system is warranted here, since the limitations below describe the Execution Layer as targeting a simulated arm. Only the actuation is virtual; the perception and control stack around it is not. The native ARCore Android implementation (Section 5.1) performs real-time visual-inertial SLAM against the device's actual camera and IMU, detects real planes in the operator's actual physical environment, and anchors the rendered arm to a real-world pose that ARCore actively re-estimates

every frame -- the same category of sensor-grounded computer vision a mapping or visual-odometry project would be evaluated on, not a scripted animation. The multi-agent decision pipeline (Section 3) and its hash-chained audit ledger (Section 3.4) are likewise real, deployed software artifacts exercised by genuine, non-scripted control input, not simulated for the purpose of this evaluation. What remains virtual is exactly one link in the chain -- the joint actuator a correct motion command is finally sent to -- which the first item below identifies as the natural next extension rather than a property of the system's evaluation methodology.

## 8. Limitations and Future Work

- Physical robot integration: the Execution Layer currently targets a simulated arm; extending the Robot Controller Interface to a physical UR5 via ROS2/MoveIt2 remains future work, as originally scoped.
- Human-subject evaluation: the User Effort Score and any claim of accessibility to non-expert operators require a real user study, not conducted here.
- PyBullet collision validation: blocked in this development environment by a macOS-SDK/toolchain incompatibility in PyBullet's bundled zlib sources; the analytic fallback is validated but is a coarser approximation than full rigid-body simulation.
- Native AR device coverage: the ARCore Android implementation (Section 5.1) has been verified functioning end-to-end on exactly one physical device via remote testing; broader compatibility across manufacturers, Android versions, and ARCore certification tiers, along with a signed release build, remains future work.
- iOS WebXR: Safari/WebKit does not implement the WebXR Device API at the time of writing, so the AR and VR session paths are Android-only; the iOS build falls back to the existing head-tracking parallax mode.
- WebXR-in-WebView generalizability: the finding that Capacitor's embedded WebView cannot execute immersive-ar sessions (Section 5.1) was established on one Android WebView version on one device; whether this holds across the full range of Android System WebView versions, or is addressable via a Trusted Web Activity wrapper instead of a full native rewrite, was not investigated, and is noted as an open question for other WebXR-based systems targeting hybrid app shells.
- Evaluation robustness: the automated benchmark's 100% completion rate reflects well-posed task design rather than stress-tested robustness; a harder adversarial and near-boundary test suite is a natural extension of the harness already built.

## 9. Conclusion

This paper documented the completion, verification, and multi-platform packaging of COGNIFORGE, a WebXR robot-teleoperation and learning-from-demonstration platform. Relative to its originating proposal, the delivered system integrates a richer nine-agent pipeline spanning reactive safety, BDI deliberation, contingency-branch motion planning with real inverse kinematics, and dual-path collision validation, all recorded through a hash-chained audit ledger. A systematic verification pass -- automated testing, static analysis, live interactive browser use, and native-shell packaging -- found and fixed ten concrete defects, several of which would have been invisible to any single verification technique alone. An automated benchmark harness replaces the proposal's placeholder evaluation figures with measurements against the real running system, explicit about which proposed metric could not honestly be reported without human subjects. The same codebase now ships as a browser application, a verified Electron desktop application, and a compiled, installed, and verified Android application for immersive VR; a second, independently verified native Android application delivers handheld AR after a further round of verification -- reported in

Section 5.1 -- found that the original WebXR-in-WebView approach to mobile AR was not viable on real hardware and required a different architecture entirely. Together, this constitutes a concrete step toward the proposal's founding claim that anyone should be able to teach a robot by simply showing it what to do, achieved across a wider and more rigorously verified range of real deployment targets than the original proposal anticipated.

## References

[1] Argall, B. D., Chernova, S., Veloso, M., & Browning, B. (2009). A survey of robot learning from demonstration. Robotics and Autonomous Systems, 57(5), 469-483.

[2] Niekum, S., Osentoski, S., Konidaris, G., & Barto, A. G. (2012). Learning and generalization of complex tasks from unstructured demonstrations. IEEE/RSJ IROS, 5239-5246.

[3] Calinon, S., Guenter, F., & Billard, A. (2010). On learning, representing, and generalizing a task in a humanoid robot. IEEE Trans. Systems, Man, and Cybernetics, Part B, 37(2), 286-298.

[4] Lipton, J., Fay, A., & Rus, D. (2018). Baxter's homunculus: Virtual reality spaces for teleoperation. IEEE Robotics and Automation Letters, 3(4), 4398-4405.

[5] Rosen, E., Whitney, D., Phillips, E., Ullman, D., & Tellex, S. (2020). Mixed reality as a bidirectional communication interface for human-robot interaction. IEEE/RSJ IROS, 11431-11438.

[6] Stone, P., & Veloso, M. (2000). Multiagent systems: A survey from a machine learning perspective. Autonomous Robots, 8(3), 345-383.

[7] Parker, L. E. (1998). ALLIANCE: An architecture for fault tolerant multirobot cooperation. IEEE Trans. Robotics and Automation, 14(2), 220-240.

[8] W3C. (2024). WebXR Device API Specification. World Wide Web Consortium.

[9] W3C. (2023). WebXR Hand Input Module -- Level 1. World Wide Web Consortium.

[10] Chase, J., & LangChain. (2023). LangGraph: Building stateful, multi-agent applications with LLMs. LangChain Blog.

[11] Multi-Agent Security literature review (2026 preprint synthesis). Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents -- Oxford Witt Lab et al., arXiv:2505.02077.

[12] G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. NeurIPS 2025.

[13] Contingency Games for Multi-Agent Interaction. IEEE Robotics and Automation Letters, 9(3), 2024.

[14] Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95), 312-319.

[15] Nakamura, Y., & Hanafusa, H. (1986). Inverse kinematic solutions with singularity robustness for robot manipulator control. Journal of Dynamic Systems, Measurement, and Control, 108(3), 163-171.

[16] Denavit, J., & Hartenberg, R. S. (1955). A kinematic notation for lower-pair mechanisms based on matrices. Journal of Applied Mechanics, 22(2), 215-221.

[17] Haber, S., & Stornetta, W. S. (1991). How to time-stamp a digital document. Journal of Cryptology, 3(2), 99-111.

[18] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547.

[19] Klein, G., & Murray, D. (2007). Parallel tracking and mapping for small AR workspaces. Proceedings of the 6th IEEE/ACM International Symposium on Mixed and Augmented Reality (ISMAR), 225-234.